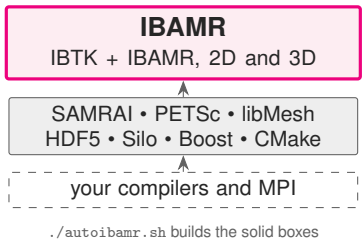
INSTALLING IBAMR WITH AUTOIBAMR

One script builds IBAMR and every library it needs

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part I • Chapter 2



Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. say what autoibamr installs and what IBAMR uses each library for;
2. choose an IBAMR version and a place to install it;
3. install the prerequisites on Ubuntu, on Windows through WSL2, on macOS, and on Puma;
4. run autoibamr, follow its progress, and read its log;
5. load an installation with `enable.sh` and test it by building an example;
6. recognize and fix the most common installation errors.

> Installing IBAMR used to take pages of notes: each library built by hand, with versions that had to match. Those recipes (2011–2019) are kept in Part X, chapter X2. This chapter replaces them.

WHAT AUTOIBAMR IS

- A shell script from the IBAMR developers that downloads, configures, builds and installs IBAMR together with the libraries it needs.
- The versions of those libraries are chosen to work with the IBAMR version you ask for.
- It runs on Linux and macOS. On Windows it runs inside WSL2, a Linux system that Windows provides.
- It is based on *candi*, the installer for the deal.II finite-element library.
- It builds IBAMR with CMake, which gives you the `IBAMRConfig.cmake` file that application builds look for (chapter 3).

Get it

https://github.com/IBAMR/autoibamr

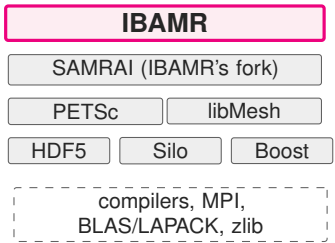
IBAMR's own instructions

https://ibamr.github.io/building

Why not the old Makefile builds?

Makefile (autotools) builds still work through IBAMR 0.19 but are removed in 0.20. Applications built with Makefiles need a `make.inc` file that autoibamr does not create.

WHAT GETS BUILT, AND WHAT EACH PIECE DOES



Library	What IBAMR uses it for
SAMRAI	adaptive Cartesian grids: patches, levels, regridding, parallel data
PETSc	vectors, matrices and the Krylov solvers and preconditioners
libMesh	finite-element meshes and assembly for IBFE
HDF5	the file format of restart files and of the fluid output in `dumps.visit`
Silo	the structure (Lagrangian) output in `lag_data.visit`
Boost	C++ utilities; bundled with IBAMR until 0.20
CMake	the build system; autoibamr downloads its own copy
numdiff	compares numbers in IBAMR's test suite

Versions installed for IBAMR 0.19.0 in October 2026: CMake 3.30.6, HDF5 1.12.2, PETSc 3.23.3, Boost 1.91.0, Silo 4.11, libMesh 1.7.8, SAMRAI 2025.10.29.

WHICH IBAMR VERSION?

IBAMR	Released	SAMRAI fork	Notes
0.16.0	31 January 2025	2025.01.09	installed on Puma for this course
0.17.0	1 October 2025	2025.09.12	small fixes
0.18.0	12 November 2025	2025.10.29	some solver classes take their settings as input databases
0.19.0	11 March 2026	2025.10.29	autoibamr's default; velocity is written only as the vector `U`
0.20	in development		CMake builds only; Boost no longer bundled

- Every example in this tutorial builds and runs with **0.16.0 and 0.19.0**; IB/13 and the three ConstraintIB examples were run with 0.19.0 only (chapter 36).
- The one difference you will notice is in VisIt: from 0.19, plot `U`, `U[0]` or `magnitude(U)` instead of `U_x`. Those expressions also work with 0.16.
- Starting fresh on a laptop? Take the default, 0.19.0.
- The autoibamr help text lists versions up to 0.17.0 only. The script itself installs 0.19.0 by default and accepts `--ibamr-version=0.18.0` and `0.19.0`.

WHERE TO INSTALL IT

Your own computer

- Linux, macOS, or Windows through WSL2.
- Good for learning, editing input files, 2D runs and VisIt.
- Needs about **6 GB** of disk: one install with its sources and build files took 5.6 GB.
- Builds in about half an hour on a fast desktop PC with 12 parallel jobs.

Puma, the UA cluster

- For 3D runs, long runs and parameter sweeps.
- Build inside a job: the compiler and MPI modules load only on compute nodes, not on the login node.
- A class or lab can share one installation in the group's folder, so each student does not spend an hour building.

> You can keep several installs side by side, one per version, with `--prefix`. Each has its own `enable.sh`; load the one that matches the version an application was built with.

PREREQUISITES ON UBUNTU (AND UBUNTU UNDER WSL2)

autoibamr needs compilers, an MPI library with its compiler wrappers, and a few tools. On Ubuntu 24.04 this one command installs everything; it was used for the tests in this tutorial:

```
sudo apt update
sudo apt install build-essential gfortran openmpi-bin libopenmpi-dev \
    python3 python3-setuptools m4 git wget curl ca-certificates \
    libtirpc-dev zlib1g-dev pkg-config libblas-dev liblapack-dev \
    unzip bzip2 xz-utils patch file
```

- `openmpi-bin` and `libopenmpi-dev` give `mpicc`, `mpicxx`, `mpif90` and `mpirun`.
- `libtirpc-dev` supplies the XDR functions libMesh needs.
- On Ubuntu 22.04, add `python3-distutils`.

Check before you start:

```
mpicc --version
mpirun --version
```

ON WINDOWS: INSTALL UBUNTU WITH WSL2

1. Open **PowerShell as administrator** and run

   ```
   wsl --install -d Ubuntu-24.04
   ```

   Restart if Windows asks, then create a Linux user name and password when Ubuntu starts for the first time.
2. Open **Ubuntu 24.04** from the Start menu and follow the Ubuntu steps on the previous slide.
3. Work in your Linux home folder (`~`), not under `/mnt/c`. Windows files are reachable at `/mnt/c/Users/<you>`, but reading and writing across that boundary is much slower, which matters for builds and simulation output.
4. From Windows, your Linux files are at `\\wsl.localhost\Ubuntu-24.04\home\<you>`, so the Windows version of VisIt can open your results directly.

Microsoft, “Install WSL” (learn.microsoft.com/windows/wsl/install).

ON MACOS

1. Install Apple's command-line tools:

```
xcode-select --install
```

2. Install an MPI library with Fortran support, for example Open MPI from Homebrew (https://brew.sh):

```
brew install open-mpi
```

Known problems on macOS (from the autoibamr README)

- Anaconda or Miniconda on your `PATH` supplies its own compilers and libraries, which conflict with the build. Remove it from `PATH` in the terminal you build in.
- Exclude the install folder from Spotlight indexing, in Spotlight's privacy settings.
- `ld: library not found for -ld_classic`: update Xcode and its command-line tools.

These macOS steps were not re-tested for this edition of the tutorial.

ON PUMA: BUILD INSIDE A BATCH JOB

Clone autoibamr on a login node (see the next slide), then submit this script with `sbatch build_ibamr.slurm`:

```
#!/bin/bash
#SBATCH --job-name=autoibamr
#SBATCH --account=YOUR_GROUP          # your PI's group name
#SBATCH --partition=standard
#SBATCH --nodes=1
#SBATCH --ntasks=16
#SBATCH --mem-per-cpu=4gb
#SBATCH --time=04:00:00
#SBATCH --output=autoibamr-%j.out

module load gnu13 openmpi5            # the modules the course install uses
export CC=mpicc CXX=mpicxx FC=mpif90
cd ~/autoibamr
./autoibamr.sh --ibamr-version=0.16.0 -j 16 --yes
```

- Leave out `--ibamr-version` to get the newest version.
- Every job that *uses* IBAMR must load the same two modules (chapter 3).

This script follows the UA HPC documentation, but it has not yet been run on Puma for this edition of the tutorial.

GET AUTOIBAMR AND RUN IT

```
cd ~
git clone https://github.com/IBAMR/autoibamr.git
cd autoibamr
export CC=mpicc CXX=mpicxx FC=mpif90      # the MPI compiler wrappers
./autoibamr.sh -j 8
```

- With no options it installs the newest IBAMR into `~/autoibamr`, the folder you cloned.
- `-j 8` compiles 8 files at a time. The default is 1, which can take many hours.
- Pick `-j` no larger than your number of cores. As a rule of thumb, allow about 2 GB of memory per job, and lower `-j` if memory runs out.
- It asks you to confirm before it starts; `--yes` answers for you.

Common choices

`--ibamr-version=0.16.0`	a specific version
`--prefix=<folder>`	install somewhere else
`-j 8` or `--jobs=8`	parallel compiling
`--yes`	no questions

MORE OPTIONS

Option	What it does
`--enable-debugging`	debug symbols and every assertion checked, no optimization; installs into `~/autoibamr-debug`. Use it to track down a crash, not for production runs.
`--enable-assertions-with-optimizations`	assertions with optimization: catches many mistakes for a 10–20% slowdown
`--enable-native-optimizations`	tunes the code for the processor it is built on. Use only when you build on the same kind of machine you run on.
`--dependencies-only`	everything except IBAMR itself
`--disable-libmesh`	no libMesh, so no IBFE
`--external-boost=<path>`	use a Boost you already have
`--lapack-library=fblaslapack`	a slower but simpler LAPACK than the default OpenBLAS, useful with profiling tools
`--enable-exodusii`	also builds NetCDF and ExodusII, which some other packages need

Run `./autoibamr.sh --help` for the full list.

HOW LONG IT TAKES

The end of the screen output from the two installs made for this tutorial (a desktop PC with 24 threads running Ubuntu 24.04 under WSL2, 12 parallel jobs):

```
IBAMR 0.16.0
Build finished in 1477 seconds.
Summary of timings:
cmake: 7 s
hdf5: 136 s
petsc: 240 s
boost: 54 s
numdiff: 10 s
silo: 22 s
libmesh: 174 s
samrai: 52 s
ibamr: 779 s
```

```
IBAMR 0.19.0
Build finished in 1730 seconds.
Summary of timings:
cmake: 6 s
hdf5: 121 s
petsc: 253 s
boost: 71 s
numdiff: 10 s
silo: 25 s
libmesh: 187 s
samrai: 63 s
ibamr: 993 s
```

- About half an hour each. IBAMR itself is more than half of the time.
- With 4 parallel jobs, expect roughly three times as long.

WHAT IS IN THE INSTALL FOLDER

```
~/autoibamr/
  autoibamr.sh  autoibamr.log      the script and its log
  configuration/
    enable.sh                      source this to use the install
    IBAMR-0.19.0  petsc-3.23.3 ... one small settings file per package
  packages/
    IBAMR-0.19.0/                  include/, lib/, lib/cmake/ibamr/IBAMRConfig.cmake
    petsc-3.23.3/  libmesh-1.7.8/  samrai-2025.10.29/  silo-4.11-bsd/ ...
    cmake-3.30.6/                  autoibamr's own copy of CMake
  tmp/
    src/                           downloaded source archives
    unpack/IBAMR-0.19.0/           IBAMR's source, including examples/
    build/v0.19.0/                 IBAMR's build folder, used to build the examples
```

> `autoibamr.log` is written in the folder you ran the script from. If a build fails, search it for the *first* error; later errors usually follow from it.

USING THE INSTALL: ENABLE.SH

Each new terminal, and each batch job, starts by loading the install:

```
source ~/autoibamr/configuration/enable.sh
```

It runs the small settings files in `configuration/`. For example, it sets these (with the full path of your home folder):

```
export IBAMR_ROOT=~/autoibamr/packages/IBAMR-0.19.0
export PETSC_DIR=~/autoibamr/packages/petsc-3.23.3
export PATH=~/autoibamr/packages/cmake-3.30.6/bin:${PATH}
```

- `IBAMR_ROOT` is how CMake finds IBAMR when you build an application (chapter 3).
- `cmake` now means autoibamr's copy, which is new enough for IBAMR.
- To load it automatically, add the `source` line to `~/.bashrc`. If you keep two versions, load each by hand instead.
- On Puma, load the modules first: `module load gnu13 openmpi5`.

TEST THE INSTALL WITH AN UPSTREAM EXAMPLE

IBAMR's own examples build inside the install's build folder:

```
source ~/autoibamr/configuration/enable.sh
cd ~/autoibamr/tmp/build/v0.19.0
make -j 4 IB-ex1                      # about half a minute
cd examples/IB/explicit/ex1
mpirun -np 4 ./main2d input2d
```

- `IB-ex1` is an elastic membrane, the upstream relative of this tutorial's rubber band (example IB/01).
- Its input files are copied next to `main2d`, so it runs from that folder.
- `make examples-IB` builds every IB example. Chapter 15 tours them.

> If this example builds and runs, the installation works. Chapter 3 builds your own programs outside this folder.

WHEN SOMETHING GOES WRONG

What you see	What to do
`CC variable not set. Please set it with $export CC=<(MPI) C compiler>`	`export CC=mpicc CXX=mpicxx FC=mpif90`, then run again
`c++: fatal error: Killed signal terminated program cc1plus`	the computer ran out of memory: lower `-j`
libMesh cannot find XDR or `rpc/rpc.h`	install `libtirpc-dev` (Ubuntu)
`mpirun has detected an attempt to run as root`	build and run as your own user, not with `sudo`
`ld: library not found for -ld_classic` on macOS	update Xcode and its command-line tools
errors that mention `anaconda` or `miniconda` paths	remove Anaconda from `PATH` in this terminal
on Puma, `module: command not found` or modules missing	you are on a login node: build inside a batch job

For anything else, find the first error in `autoibamr.log` and ask on the IBAMR discussion forum: https://github.com/IBAMR/IBAMR/discussions.

SUMMARY

1. autoibamr builds IBAMR and matching versions of SAMRAI, PETSc, libMesh, HDF5, Silo and Boost in one command.
2. Install the prerequisites first: compilers, MPI with `mpicc`/`mpicxx`/`mpif90`, and a few tools.
3. Run `./autoibamr.sh -j N`. Always give `-j`; add `--ibamr-version` for an older version.
4. On Puma, build inside a batch job after `module load gnu13 openmpi5`.
5. Load an install with `source .../configuration/enable.sh`, which sets `IBAMR_ROOT`.
6. Test it by building `IB-ex1` in `tmp/build/v<version>`.

Next, chapter 3: build and run your own IBAMR program with CMake, starting with the rubber band.

REFERENCES

- IBAMR developers. *autoibamr*. https://github.com/IBAMR/autoibamr (commit `9c1e0a5`, October 2026).
- IBAMR developers. *Building IBAMR*. https://ibamr.github.io/building.
- IBAMR developers. *Building IBAMR applications*. https://ibamr.github.io/applications.
- deal.II developers. *candi: compile and install*. https://github.com/dealii/candi.
- Microsoft. *Install WSL*. https://learn.microsoft.com/windows/wsl/install.
- The tests in this chapter used Ubuntu 24.04.5 under WSL2 with GCC 13.3.0 and Open MPI 4.1.6.